End Lab 00:42:03

Caution: When you are in the console, do not deviate from the lab instructions. Doing so may cause your account to be blocked. Learn more.

Open Google Cloud Console

Username

student-02-0ad1fc587afc

Password

ODuQpQfeP6vW

GCP Project ID

qwiklabs-gcp-00-5c403b2c

Introduction to Function Calling with Gemini

Lab 1 hour No cost Intermediate

Rate Lab

This lab may incorporate AI tools to support your learning.

Lab instructions and tasks

GENAI036

Gemini

Gemini is a family of generative AI models developed by Google DeepMind that is designed for multimodal use cases. The Gemini API gives you access to the Gemini Pro and Gemini Flash models.

Calling functions from Gemini

The Vertex AI Gemini API provides a unified interface for interacting with Gemini models.

Function calling lets developers create a description of a function in their code, then pass that description to a language model in a request. The response from the model includes the name of a function that matches the description and the arguments to call it with.

Function calling is similar to Vertex AI Extensions in that they both generate information about functions. The difference between them is that function calling returns JSON data with the name of a function and the arguments to use in your code, whereas Vertex AI Extensions returns the function and calls it for you.

Why function calling?

Imagine asking someone to write down important information without giving them a form or any guidelines on the structure. You might get a beautifully crafted paragraph, but extracting specific details like names, dates, or numbers would be tedious! Similarly, trying to get consistent structured data from a generative text model without function calling can be frustrating. You're stuck explicitly prompting for things like JSON output, often with inconsistent and frustrating results.

This is where Gemini Function Calling comes in. Instead of hoping for the best in a freeform text response from a generative model, you can define clear functions with specific parameters and data types. These function declarations act as structured guidelines, guiding the Gemini model to structure its output in a predictable and usable way. No more parsing text responses for important information!

Think of it like teaching Gemini to speak the language of your applications. Need to retrieve information from a database? Define a `search_db` function with parameters for search terms. Want to integrate with a weather API? Create a `get_weather` function that takes a location as input. Function calling bridges the gap between human language and the structured data needed to interact with external systems.

Objectives

Objectives

In this lab, you will learn how to use the Vertex AI Gemini API with the Vertex AI SDK for Python to make function calls via the Gemini Flash (`gemini-2.0-flash`) model.

You will complete the following tasks:

- Install the Vertex AI SDK for Python
- Use the Vertex AI Gemini API to interact with the Gemini Flash (gemini-2.0-flash) model:
    - Generate function calls from a text prompt to get the weather for a given location.
    - Generate function calls from a text prompt and call an external API to geocode addresses.
    - Generate function calls from a chat prompt to help retail users.

Setup

Before you click the Start Lab button

Read these instructions. Labs are timed and you cannot pause them. The timer, which starts when you click **Start Lab**, shows how long Google Cloud resources will be made available to you.

This Qwiklabs hands-on lab lets you do the lab activities yourself in a real cloud environment, not in a simulation or demo environment. It does so by giving you new, temporary credentials that you use to sign in and access Google Cloud for the duration of the lab.

What you need

To complete this lab, you need:

- Access to a standard internet browser (Chrome browser recommended).
- Time to complete the lab.

Note: If you already have your own personal Google Cloud account or project, do not use it for this lab.

Note: If you are using a Pixelbook, open an Incognito window to run this lab.

How to start your lab and sign in to the Google Cloud console

1. Click the **Start Lab** button. If you need to pay for the lab, a dialog opens for you to select your payment method. On the left is the Lab Details pane with the following:
    - The Open Google Cloud console button

- Time remaining
- The temporary credentials that you must use for this lab
- Other information, if needed, to step through this lab

2. Click **Open Google Cloud console** (or right-click and select **Open Link in Incognito Window** if you are running the Chrome browser).

   The lab spins up resources, and then opens another tab that shows the Sign in page.

   Tip: Arrange the tabs in separate windows, side-by-side.

   > **Note:** If you see the **Choose an account** dialog, click **Use Another Account**.

3. If necessary, copy the **Username** below and paste it into the **Sign in** dialog.

   ```
   student-02-0ad1fc587afc@qwiklabs.net
   ```

   You can also find the Username in the Lab Details pane.

4. Click **Next**.

5. Copy the **Password** below and paste it into the **Welcome** dialog.

   ```
   ODuQpQfeP6vW
   ```

   You can also find the Password in the Lab Details pane.

6. Click **Next**.

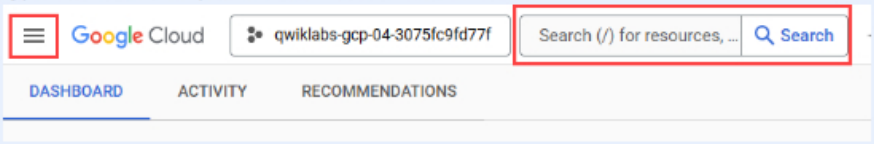
   > **Important:** You must use the credentials the lab provides you. Do not use your Google Cloud account credentials.

   > **Note:** Using your own Google Cloud account for this lab may incur extra charges.

7. Click through the subsequent pages:
   - Accept the terms and conditions.
   - Do not add recovery options or two-factor authentication (because this is a temporary account).
   - Do not sign up for free trials.

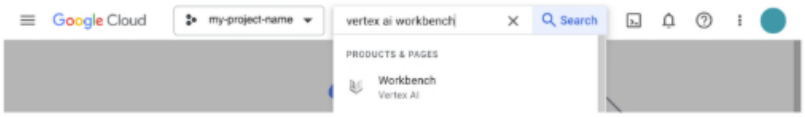
After a few moments, the Google Cloud console opens in this tab.

Note: To access Google Cloud products and services, click the **Navigation menu** or type the service or product name in the **Search** field.



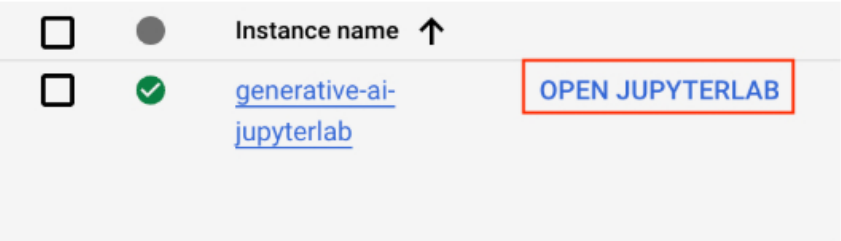
Task 1. Open Python Notebook and Install Packages

1. In the Google Cloud console, navigate to **Vertex AI Workbench**. In the top search bar of the Google Cloud console, enter **Vertex AI Workbench**, and click on the first result.



2. Under **Instances**, click on **Open JupyterLab** for **generative-ai-jupyterlab** instance.

   The JupyterLab will run in a new tab.



3. On the **Launcher**, under **Notebook**, click on **Python 3** to open a new python notebook.

4. Install Vertex AI SDK for Python by the running the following command in the first cell of the notebook. Either click the play ▶ button at the top or enter **SHIFT+ENTER** on your keyboard to execute the cell.

```
! pip install --upgrade --quiet google-genai
```

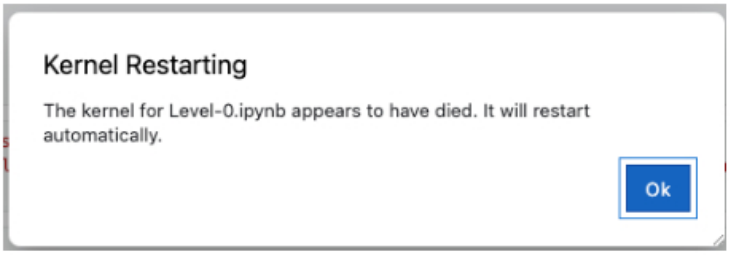
Output:

5. To use the newly installed packages in this Jupyter runtime, it is recommended to restart the runtime. Restart the kernel by running the below code snippet or clicking the refresh button at the top, followed by clicking **Restart** button.

```
import IPython

app = IPython.Application.instance()
app.kernel.do_shutdown(True)
```

Output:



After the restart is complete, click **Ok** on the prompt to continue.

6. Set the Google Cloud project information.

```
import os

PROJECT_ID = "qwiklabs-gcp-00-5c403b2c290d"
if not PROJECT_ID or PROJECT_ID == "[your-project-id]":
    PROJECT_ID = str(os.environ.get("GOOGLE_CLOUD_PROJECT"))

LOCATION = os.environ.get("GOOGLE_CLOUD_REGION", "us-central1")

from google import genai

client = genai.Client(vertexai=True, project=PROJECT_ID,
location=LOCATION)
```

7. Run the following code snippet in the next cell to import the libraries.

```
from IPython.display import Markdown, display
from google.genai.types import FunctionDeclaration,
GenerateContentConfig, Part, Tool
import requests
```

Task 2. Function Calling to geocode addresses with a maps API

Let's generate a function call that has a more complex structure. In this task, you'll define a function that takes multiple parameters as inputs. Then you'll use automatic function calling in the Gemini API to make a live API call to convert an address to latitude and longitude coordinates.

The Gemini Flash (`gemini-2.0-flash`) model is designed to handle natural language tasks, multiturn text and code chat, and code generation.

1. Load the Gemini Flash model:

```
MODEL_ID = "gemini-2.0-flash-001"
```

2. Run the following code snippet:

```
def get_location(
    amenity: str | None = None,
    street: str | None = None,
    city: str | None = None,
    county: str | None = None,
    state: str | None = None,
    country: str | None = None,
    postalcode: str | None = None,
) -> list[dict]:
    """
    Get latitude and longitude for a given location.

    Args:
        amenity (str | None): Amenity or Point of interest.
        street (str | None): Street name.
        city (str | None): City name.
        county (str | None): County name.
        state (str | None): State name.
        country (str | None): Country name.
        postalcode (str | None): Postal code.

    Returns:
        list[dict]: A list of dictionaries with the latitude and
longitude of the given location.
                    Returns an empty list if the location cannot
be determined.
    """
    base_url = "https://nominatim.openstreetmap.org/search"
    params = {
        "amenity": amenity,
        "street": street,
        "city": city,
        "county": county,
        "state": state,
        "country": country,
        "postalcode": postalcode,
        "format": "json",
    }
    # Filter out None values from parameters
```

```
    params = {k: v for k, v in params.items() if v is not None}

    try:
        response = requests.get(base_url, params=params,
headers={"User-Agent": "none"})
        response.raise_for_status()
        return response.json()
    except requests.RequestException as e:
        print(f"Error fetching location data: {e}")
        return []
```

3. Next, call the model to generate a response.

```
prompt = """
I want to get the coordinates for the following address:
1600 Amphitheatre Pkwy, Mountain View, CA 94043
"""

response = client.models.generate_content(
    model=MODEL_ID,
    contents=prompt,
    config=GenerateContentConfig(tools=[get_location],
temperature=0),
)
print(response.text)
```

Output:

```
The coordinates for 1600 Amphitheatre Pkwy, Mountain View, CA 94043 are: l
```

Great work! You were able to construct a function and tool that the LLM used to output the parameters necessary for a function call, then you actually made the function call to obtain the coordinates of the specified location.

Here we used the OpenStreetMap Nominatim API to geocode an address to make it easy to use and learn in this notebook. If you're working with large amounts of maps or geolocation data, you can use the Google Maps Geocoding API.

Task 3. Function calling in a chat session

Next, let's use the chat model in Gemini to help customers get information about products in a store.

1. Let's start by defining multiple functions within a tool for getting the product information, the location of stores and placing an order.

```
get_product_info = FunctionDeclaration(
    name="get_product_info",
    description="Get the stock amount and identifier for a given
```

```
product",
    parameters={
        "type": "OBJECT",
        "properties": {
            "product_name": {"type": "STRING", "description":
"Product name"}
        },
    },
)

get_store_location = FunctionDeclaration(
    name="get_store_location",
    description="Get the location of the closest store",
    parameters={
        "type": "OBJECT",
        "properties": {"location": {"type": "STRING",
"description": "Location"}},
    },
)

place_order = FunctionDeclaration(
    name="place_order",
    description="Place an order",
    parameters={
        "type": "OBJECT",
        "properties": {
            "product": {"type": "STRING", "description":
"Product name"},
            "address": {"type": "STRING", "description":
"Shipping address"},
        },
    },
)

retail_tool = Tool(
    function_declarations=[
        get_product_info,
        get_store_location,
        place_order,
    ],
)
```

2. Function calling can also be used in a multi-turn chat session. You can specify tools when creating a model to avoid having to send them with every request.

```
chat = client.chats.create(
    model=MODEL_ID,
    config=GenerateContentConfig(
        temperature=0,
        tools=[retail_tool],
    ),
)
```

3. Start the conversation by asking if they have a certain product in stock.

```
prompt = """
Do you have the Pixel 9 in stock?
"""

response = chat.send_message(prompt)
response.function_calls[0]
```

Output:

```
FunctionCall(
  args={
    'product_name': 'Pixel 9'
  },
  name='get_product_info'
)
```

As expected, the response includes a structured function call that we can use to communicate with external systems.

In reality, you would execute function calls against an external system or database. Since this lab focuses on the ability to extract function parameters and generate function calls, you'll use mock data to feed responses back to the model rather than using an actual API server.

4. Use synthetic data to simulate a response payload from an external API.

```
api_response = {"sku": "GA04834-US", "in_stock": "yes"}
```

5. Let's include details from the external API call and generate a response to the user.

```
response = chat.send_message(
    Part.from_function_response(
        name="get_product_info",
        response={
            "content": api_response,
        },
    ),
)
display(Markdown(response.text))
```

Output:

```
Yes, we have the Pixel 9 in stock. Its SKU is GA04834-US.
```

6. The user might ask where they can buy a phone from a nearby store.

```
prompt = """
```

```
What about the Pixel 9 Pro XL? Is there a store in
Mountain View, CA that I can visit to try one out?
"""

response = chat.send_message(prompt)
response.function_calls
```

Output:

```
[FunctionCall(
   args={
     'product_name': 'Pixel 9 Pro XL'
   },
   name='get_product_info'
 ),
 FunctionCall(
   args={
     'location': 'Mountain View, CA'
   },
   name='get_store_location'
 )]
```

We get a response with another structured function call, this time it's set up to use a different function from our tool.

7. Use synthetic data to simulate a response payload from an external API.

```
product_info_api_response = {"sku": "GA08475-US", "in_stock":
"yes"}
store_location_api_response = {
    "store": "2000 N Shoreline Blvd, Mountain View, CA 94043,
US"
}
```

8. Again, let's include details from the external API call and generate a response to the user.

```
response = chat.send_message(
    [
        Part.from_function_response(
            name="get_product_info",
            response={
                "content": product_info_api_response,
            },
        ),
        Part.from_function_response(
            name="get_store_location",
            response={
                "content": store_location_api_response,
            },
        ),
    ]
)
```

```
display(Markdown(response.text))
```

Output:

```
Yes, we have the Pixel 9 Pro XL in stock, SKU GA08475-US. The store closes
```

9. Finally, the user might ask to order a phone and have it shipped to an address.

```
prompt = """
I'd like to order a Pixel 9 Pro XL and have it shipped to 1155
Borregas Ave, Sunnyvale, CA 94089.
"""

response = chat.send_message(prompt)
response.function_calls
```

Output:

```
[FunctionCall(
   args={
     'address': '1155 Borregas Ave, Sunnyvale, CA 94089',
     'product': 'Pixel 9 Pro XL'
   },
   name='place_order'
 )]
```

Perfect! We extracted the user's desired product, address, fetched their account number, and now we can call an API to place the order:

10. Use synthetic data to simulate a response payload from an external API.

```
order_api_response = {
    "payment_status": "paid",
    "order_number": 12345,
    "est_arrival": "2 days",
}
```

11. Let's include details from the external API call and generate a response to the user.

```
response = chat.send_message(
    Part.from_function_response(
        name="place_order",
        response={
            "content": order_api_response,
        },
    ),
)
```

```
display(Markdown(response.text))
```

Output:

```
OK. I've placed an order for a Pixel 9 Pro XL to be shipped to 1155 Borrega
```

Congratulations!

You successfully had a multi-turn conversation with Gemini, generated function calls, handled passing (mock) data back to the model, and generated messages that made use of the function responses.

Manual Last Updated July 14, 2025

Lab Last Tested July 14, 2025